



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

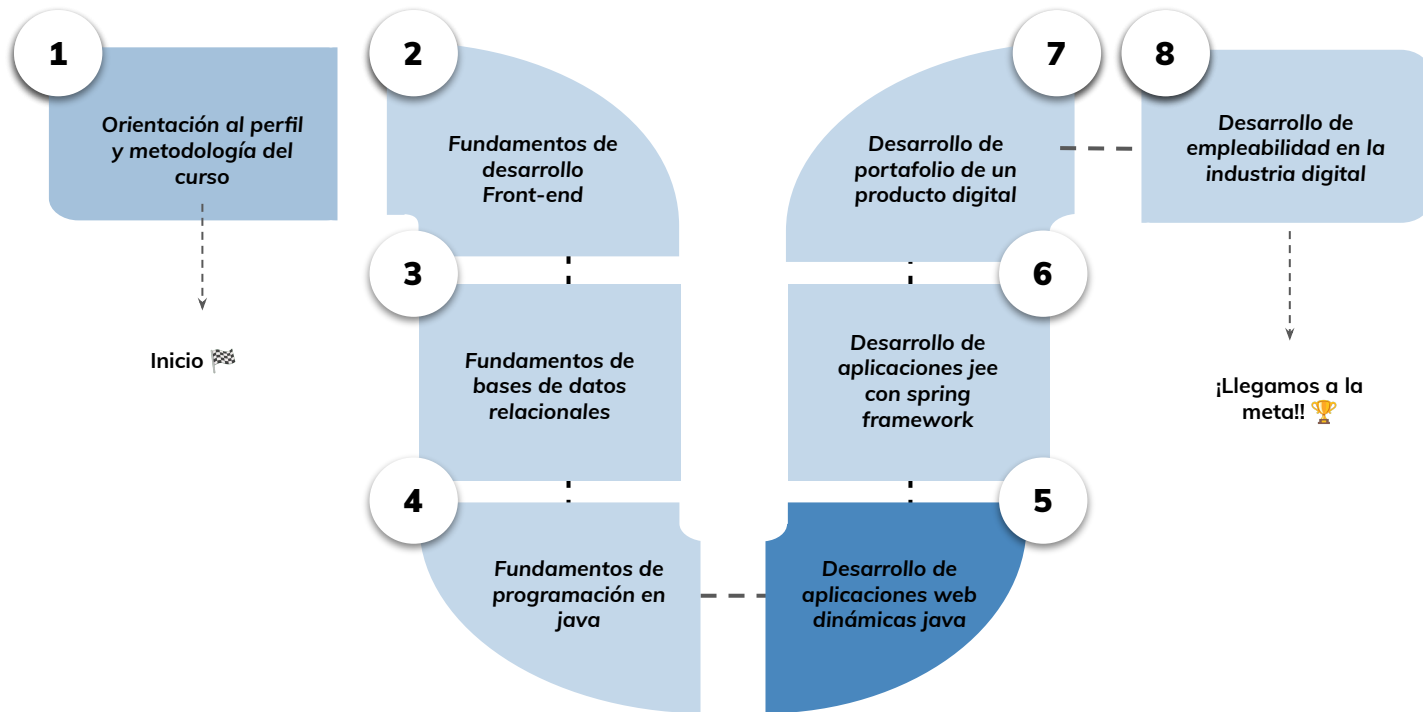


› Java server pages - Parte II

Aprendizaje Esperado 2: Implementar capa de vista de una aplicación web dinámica utilizando java server pages de acuerdo a las especificaciones entregadas.

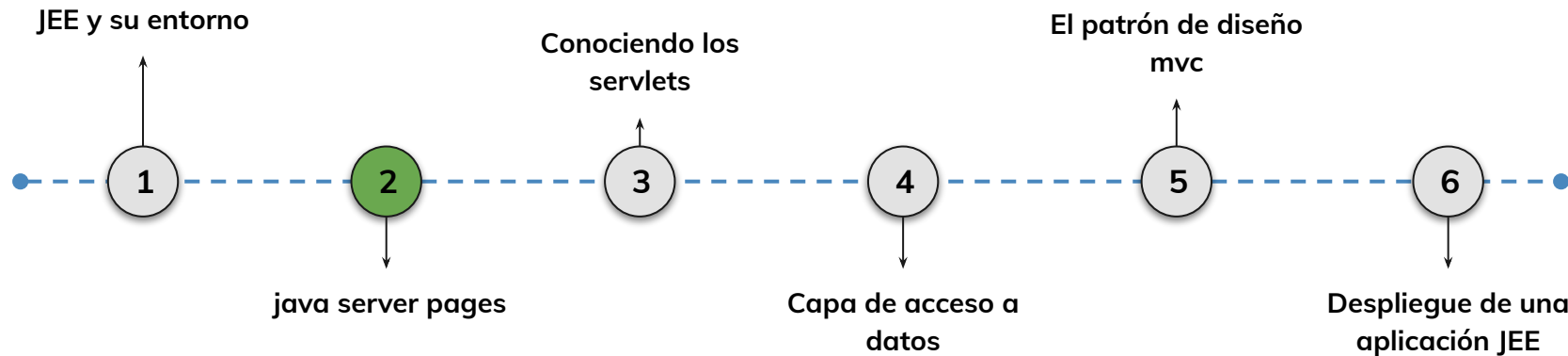
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

2.

Java Server Pages

- Iterando con `c:forEach`
- Utilizando funciones útiles en JSTL
- Creando formularios para capturar información

Tag `c:forEach`

`C:FOREACH`

Creación de formulario

Formulario



Objetivos de aprendizaje

¿Qué aprenderás?



- Reconocer la implementación de la etiqueta `c:forEach`
- Comprender el uso de las funciones en JSTL
- Aprender a crear formularios de ingresos de datos y capturar la información.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Comprender el concepto e implementación de JSP
- Reconocer el uso y configuración de JSTL
- Comprender la utilización de las etiquetas `c:out`, `c:if` y `c:choose`

Iterando con `<c:forEach>`

< > Iterando con <c:forEach>

¿Para qué sirve?:

Se utiliza para **realizar iteraciones sobre una colección de elementos** en una página JSP. Permite recorrer listas, arrays, mapas y otros tipos de colecciones de datos.

La **sintaxis** básica de <c:forEach> es la siguiente:

```
<c:forEach items ="colección" var="elemento" [otros atributos opcionales]>  
  <!-- Código a ejecutar por cada elemento de la colección -->  
</c:forEach >
```



< > Iterando con <c:forEach>

Estas etiquetas se utilizan como una buena **alternativa para** incrustar un **bucle while, do- while o for** de Java a través de un scriptlet. La etiqueta <c:forEach> es la etiqueta más utilizada porque itera sobre una colección de objetos.

Veamos un ejemplo: en este caso, le estamos dando parámetros de comienzo y finalización.

```
<body>
<c:forEach var="j" begin="1" end="3">
  Item <c:out value="${j}"/> <p>
</c:forEach>
</body>
```

```
Item 1
Item 2
Item 3
```

Con la etiqueta <c:out> estamos imprimiendo 'Item + el valor de j' en cada iteración.



LIVE DEMO

Probando, probando:

En este ejemplo vamos probar todas las etiquetas JSTL que hemos aprendido. Vamos a setear algunos valores para que podamos realizar todo el ejercicio desde la página JSP. En este caso, simularemos algunos importes + IVA.

Comenzaremos con las siguientes líneas:

```
<c:set value="500" var="importe" />
```

```
<c:out value="${importe}" />
```

```
<c:set value="Profe" var="nombre" />
```

```
<c:out value="${nombre}" />
```

LIVE DEMO

A continuación vamos a calcular el IVA:

```
<c:set value="${importe*.21}" var="iva" />
<c:out value="${iva}" />
<c:if test="${iva > 100}">
    <p>¡Que caro!</p>
</c:if>
```

LIVE CODING

Luego, vamos a mostrar mensajes por pantalla según la evaluación:

```
<c:choose>
  <c:when test="${iva < 100}">
    <c:out value="${iva} menor de 100" />
  </c:when>
  <c:when test="${iva < 200}">
    <c:out value="${iva} menor de 200" />
  </c:when>
  <c:otherwise>
    <c:out value="${iva} mayor de 200" />
  </c:otherwise>
</c:choose>
```

LIVE CODING

Finalmente, recorreremos los items y mostramos lo iterado:

```
<c:forEach var="i" items="1,4,5,6,7,8,9">  
Item <c:out value="Nº ${i}" />  
    <p>  
</c:forEach>
```

Tiempo: 25 minutos

Funciones JSTL

Funciones JSTL

¿Para qué sirven?:

Se utilizan para realizar **toma de decisiones condicionales** en las páginas JSP. Estas etiquetas **permiten controlar el flujo de ejecución** en función de condiciones específicas.

El tag **<c:if>** se utiliza para evaluar una condición y ejecutar un bloque de código si la condición es verdadera.

Por otro lado, el tag **<c:choose>** se utiliza cuando se necesita realizar múltiples tomas de decisiones y ejecutar diferentes bloques de código en función de varias condiciones.

Funciones JSTL

JSTL dispone de un conjunto de funciones que pueden emplearse desde dentro del lenguaje de expresiones, y permiten sobre todo manipular cadenas, para sustituir caracteres, concatenar, etc.

Para utilizar estas funciones, debemos cargar la directiva taglib correspondiente:

```
<%@ taglib uri="http://java.sun.com/jstl/ea/functions" prefix="fn" %>
```

Y luego utilizamos las funciones que hay disponibles dentro de una expresión del lenguaje:

La cadena tiene `<c:out value="${fn:length(miCadena)}"/>` caracteres

Funciones JSTL

Algunas de sus funciones útiles son:

- **fn:length()** : Devuelve la longitud de una cadena, una lista, un array o una colección.
- **fn:toUpperCase()** y **fn:toLowerCase()** : Convierten una cadena a mayúsculas o minúsculas, respectivamente.
- **fn:substring(texto,inicio,final)** : Devuelve una subcadena de una cadena dada, especificando el índice de inicio y, opcionalmente, el índice de finalización.
- **fn:replace(texto, palabra1,palabra2)** : Reemplaza todas las apariciones de una subcadena por otra en una cadena dada.
- **fn:contains(var, itema buscar)** : Verifica si una cadena contiene otra cadena dada y devuelve un valor booleano.

LIVE CODING

Siguiendo en el ejercicio anterior: vamos a cambiar el nombre 'Profe' a mayúsculas, y reemplazar la letra "E" por un '3'

Luego, vamos a evaluar si la cadena contiene un caracter '3' en sí misma. Si esto es verdadero, debemos mostrar por pantalla "Nombre inválido"

Tiempo: 15 minutos



Ejercicio N° 1

C : FOREACH

C : FOREACH

Manos a la obra: 🙌

Vamos a poner en práctica lo aprendido editando el archivo index.jsp del proyecto AlkeWallet.

Consigna: 📝

Vamos a calcular el interés de diversos medios de pago. En caso de que el medio elegido sea Efectivo, el interes será 0. Por otro lado el medio de pago tarjeta tiene un interés del 10%. Finalmente, el medio de pago depósito tiene un interés de 20%.

2. Cargar una lista con los 3 medios de pago y mostrar por pantalla el interés final de cada uno. El valor inicial será el que decidas.

Tiempo 🕒: 20 minutos

Formularios

< > Formularios

¿Qué son los formularios HTML?:

Los formularios son una de las herramientas de que disponemos a la hora de hacer nuestras páginas Web interactivas, en el sentido de que nos permiten recopilar información de la persona que ve la página, procesarla y responder a ella, pudiendo de esta forma responder adecuadamente a sus acciones o peticiones.

El resultado de cualquier formulario es una lista de variables y valores asignados a las mismas, que tienen todos ellos un atributo en común: el nombre de su variable.



< > Formularios

En JSP:

Si bien el formulario es de HTML, lo que haremos con JSP es procesar la información que nos llega desde el formulario y capturarla para poder manipularla.

Vamos a poner en práctica para comprender mejor



LIVE CODING

Trabajaremos sobre un archivo index.jsp.

Es momento de crear un formulario para ingreso de datos. En la etiqueta <body> del archivo index.jsp, vamos a crear a modo de ejemplo el formulario que vemos en la imagen.

```
<body>
<h1>Formulario</h1>

<form action="procesar-formulario.jsp" method="POST">
  <label for="nombre">Nombre:</label>
  <input type="text" id="nombre" name="nombre" /><br/>

  <label for="edad">Edad:</label>
  <input type="number" id="edad" name="edad" /><br/>

  <input type="submit" value="Enviar" />
</form>
```

LIVE CODING

Como se puede apreciar, es un formulario simple con inputs de ingreso de nombre y edad, además del botón submit.

Según el **atributo action** que tenemos declarado, cuando se envía el formulario, **los datos ingresados en los campos "Nombre" y "Edad" se envían a la página "procesar-formulario.jsp"** para su procesamiento.

Vamos a crear este archivo jsp también en la carpeta webapp.

```
<body>
<h1>Formulario</h1>

<form action="procesar-formulario.jsp" method="POST">
  <label for="nombre">Nombre:</label>
  <input type="text" id="nombre" name="nombre" /><br/>

  <label for="edad">Edad:</label>
  <input type="number" id="edad" name="edad" /><br/>

  <input type="submit" value="Enviar" />
</form>
```

LIVE CODING

Archivo procesar-formulario.jsp:

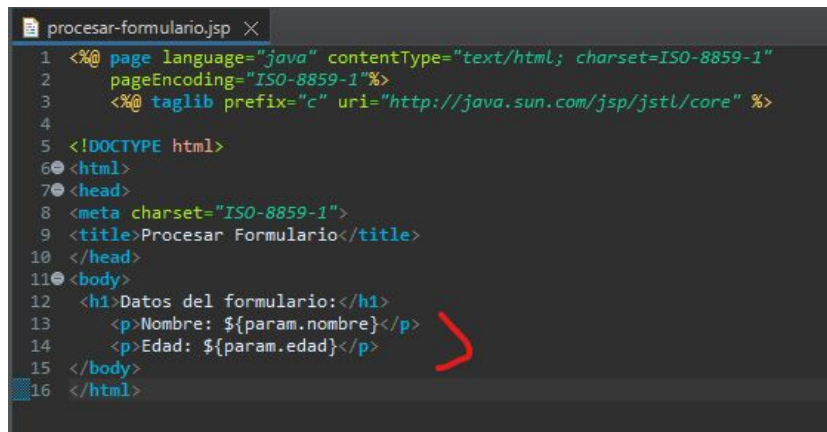
Como dijimos anteriormente, cuando se envía el formulario del index, los datos ingresados en los campos "Nombre" y "Edad" se envían a la página "procesar-formulario.jsp" para su procesamiento.

```
procesar-formulario.jsp X
1  <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2     pageEncoding="ISO-8859-1"%>
3     <%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
4
5  <!DOCTYPE html>
6  <html>
7  <head>
8     <meta charset="ISO-8859-1">
9     <title>Procesar Formulario</title>
10  </head>
11  <body>
12     <h1>Datos del formulario:</h1>
13     <p>Nombre: ${param.nombre}</p>
14     <p>Edad: ${param.edad}</p>
15  </body>
16  </html>
```

LIVE CODING

Recuerda que en la página "**procesar-formulario.jsp**", debes utilizar los **mismos nombres de parámetros (nombre y edad)** para acceder a los valores enviados desde el formulario utilizando el objeto **param**.

De esta manera, al ejecutar el proyecto, podremos ver primero el archivo `index.jsp` con el formulario, y al presionar **submit** se abrirá el archivo `procesar-formulario.jsp` recuperando los datos enviados y mostrando por pantalla.



```
1 <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2   pageEncoding="ISO-8859-1"%>
3   <%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
4
5   <!DOCTYPE html>
6   <html>
7   <head>
8     <meta charset="ISO-8859-1">
9     <title>Procesar Formulario</title>
10  </head>
11  <body>
12    <h1>Datos del formulario:</h1>
13    <p>Nombre: ${param.nombre}</p>
14    <p>Edad: ${param.edad}</p>
15  </body>
16 </html>
```



Ejercicio N° 1

Formulario

Formulario

Manos a la obra: 🙌

Vamos a editar el archivo `index.jsp` para incluir un formulario HTML (el formulario se encuentra en los siguientes slides).

Consigna: ✍️

Crear una página llamada **proceso.jsp** (también en la carpeta `webapp`), donde deberás mostrar:

- a. el nombre del usuario.
- b. la operación seleccionada
- c. el tipo de moneda elegida.

Tiempo 🕒: 40 minutos.

Formulario

```
<form action="proceso.jsp" method="post">  
  Usuario:  
  <input type="text" name="nombre">  
  Contraseña:  
  <input type="password" name="password">
```

a. el nombre del usuario.

¿Qué desea hacer? :

```
<select name="operacion">  
  <option value="retirar"> Retirar Dinero  
  <option value="ingresar" selected> Ingresar Dinero  
  <option value="consultar"> Consultar Saldo  
</select>
```

b. la operación
seleccionada

/ sigue en el proximo slide /

Formulario

c. el tipo de moneda elegida.

```
<input type="Radio" name="moneda" value="CLP"checked>Peso Chileno  
<br/>  
<input type="Radio" name="moneda" value="USD"> Dolar Estadounidense  
<br/>  
<input type="Radio" name="moneda" value="EUR"> Euro  
<br/>  
<input type="Radio" name="moneda" value="JPY"> Yen  
<br/>
```

```
<p><input type="submit" value="Enviar"></p>
```

```
</form>
```




Momento:

Time-out!



5 -10 min.



¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ **Comprender la utilización de la etiqueta `c:forEach`**
- ✓ **Entender la implementación de las funciones JSTL**
- ✓ **Comprender la captura de datos de un formulario utilizando JSP.**



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

